```
import numpy as np

# --- 1. SIMULATE AN MRI SCAN HISTOGRAM ---
# Contains 3 distinct tissue peaks: Background (low), Healthy (mid), Tumor (high)
np.random.seed(42)
mock_scan = np.hstack([
    np.random.randint(10, 60, 400),    # Background pixels
    np.random.randint(100, 160, 500),  # Healthy tissue pixels
    np.random.randint(200, 250, 300)   # Bright tumor pixels
])

# Process image data into pixel distribution probabilities
counts, _ = np.histogram(mock_scan, bins=256, range=(0, 256))
probabilities = counts / len(mock_scan)

# --- 2. FITNESS FUNCTION (Otsu's Variance) ---
def calculate_fitness(thresholds):
    """Measures boundary quality. Higher score = cleaner tissue separation."""
    thresh = np.sort(np.round(thresholds).astype(int))
    thresh = np.clip(thresh, 1, 254)

    # Slice the image graph into separate tissue zones based on the thresholds
    split_indices = np.searchsorted(np.arange(256), thresh)
    regions = np.split(probabilities, split_indices)

    global_mean = np.sum(np.arange(256) * probabilities)
    between_class_variance = 0

    for i, region in enumerate(regions):
        weight = np.sum(region)
        if weight == 0: continue

        start = 0 if i == 0 else thresh[i-1]
        end = 256 if i == len(regions)-1 else thresh[i]

        mean = np.sum(np.arange(start, end) * region) / weight
        between_class_variance += weight * (mean - global_mean) ** 2

    return between_class_variance

# --- 3. LÉVY FLIGHT MOTION ---
def levy_step(shape):
    """Generates a mix of tiny steps and giant leaps."""
    u = np.random.normal(0, 1, size=shape)
    v = np.random.normal(0, 1, size=shape)
    return u / (np.abs(v) ** (1 / 1.5))

# --- 4. CUCKOO OPTIMIZATION LOOP ---
num_nests = 15
num_thresholds = 2
pa = 0.25  # Probability of a host discovering a bad egg

# Initialize random initial boundary lines
nests = np.random.randint(50, 200, size=(num_nests, num_thresholds))
fitness = np.array([calculate_fitness(n) for n in nests])

# Run optimization cycles
for iteration in range(30):
    best_nest = nests[np.argmax(fitness)]

    # Move the cuckoos using Levy flights (Exploration phase)
    new_nests = nests + 0.5 * levy_step(nests.shape) * (nests - best_nest)
    new_nests = np.clip(np.round(new_nests).astype(int), 1, 254)

    # Evaluate new positions and update nests if they got better contrast
    for i in range(num_nests):
        new_fit = calculate_fitness(new_nests[i])
        if new_fit > fitness[i]:
            nests[i] = new_nests[i]
            fitness[i] = new_fit

    # Abandonment phase: Discard the worst 25% of boundary sets
    worst_idx = np.argsort(fitness)[:int(pa * num_nests)]
    for idx in worst_idx:
        nests[idx] = np.random.randint(10, 240, size=num_thresholds)
```

```
        fitness[idx] = calculate_fitness(nests[idx])

# Output the perfect separation lines found
final_best_nest = np.sort(nests[np.argmax(fitness)])
print(f"Optimal Tissue Separation Lines Found at Pixel Brightness: {final_best_nest}")
# Output will display the two perfect cut points separating the three tissue zones.
```

```
Optimal Tissue Separation Lines Found at Pixel Brightness: [ 60 173]
```